

VDR Toy LLM

Exact Rational Arithmetic in a Transformer Language Model Using Fixed-Denominator VDR Triples

Registry: [\[HOWL-VDR-31-2026\]](#)

Series Path: [\[HOWL-VDR-1-2026\]](#) → [\[HOWL-VDR-2-2026\]](#) → [\[HOWL-MATH-3-2026\]](#) → [\[HOWL-MATH-4-2026\]](#) → ... → [\[HOWL-VDR-14-2026\]](#) → ... → [\[HOWL-VDR-21-2026\]](#) → [\[HOWL-VDR-22-2026\]](#) → [\[HOWL-VDR-23-2026\]](#) → [\[HOWL-VDR-24-2026\]](#) → [\[HOWL-VDR-25-2026\]](#) → [\[HOWL-VDR-26-2026\]](#) → [\[HOWL-VDR-27-2026\]](#) → [\[HOWL-VDR-28-2026\]](#) → [\[HOWL-VDR-29-2026\]](#) → [\[HOWL-VDR-30-2026\]](#) → [\[HOWL-VDR-31-2026\]](#)

DOI: 10.5281/zenodo.20276904

Date: May 2026

Domain: ML Infrastructure Economics / Exact Arithmetic

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic’s Opus 4.6.

Abstract

We present a complete transformer language model — embedding, self-attention, feed-forward network, backpropagation, weight updates, and text generation — running entirely in exact rational arithmetic with no floating-point operations. The model uses the `vdr-math` Python library, which represents every value as an ordered triple $[V, D, R]$ (Value, Denominator, Remainder) with a fixed denominator $D = 2^{32}$. We describe the denominator growth problem that arises when standard rational arithmetic operators are applied in a fixed-frame system, the operator-level solution we implemented to prevent it, and the quadratic softmax surrogate that eliminates transcendental functions from the forward pass. The resulting toy model — 181 parameters, vocabulary of 5, trained for 20 epochs on a 6-word corpus — passes 9 verification tests including bit-identical determinism across runs and exact sum-to-one softmax outputs. All denominators remain at 2^{32} through every operation chain. The work demonstrates that a fixed-denominator rational arithmetic system can support a complete LLM training and inference pipeline, and identifies the engineering steps required to scale beyond toy dimensions.

1. Introduction

Floating-point arithmetic is the universal substrate of modern machine learning. Every weight, activation, gradient, and probability in a neural network is a 16-bit, 32-bit, or 64-

bit IEEE 754 float. This representation is fast and hardware-supported, but it introduces three structural problems.

First, every operation rounds. The mantissa is finite, the rounding is invisible, and the error compounds across operation chains. A single forward pass through a 96-layer transformer involves millions of multiply-accumulate operations, each contributing a rounding error on the order of 10^{-7} (float32) or 10^{-16} (float64). These errors accumulate and can become indistinguishable from the signal in long chains — diffusion model frame generation, Kalman filter cycles, or financial risk calculations.

Second, floating-point addition is not associative. The expression $(a + b) + c$ does not in general produce the same bit pattern as $a + (b + c)$. This means that distributed training across multiple GPUs — where gradient reductions may occur in different orders depending on thread scheduling — produces nondeterministic results. Two training runs from the same seed on the same hardware can diverge after a few hundred steps.

Third, the rounding is silent. No operation reports how much information was discarded. There is no error budget, no accumulated-loss counter, no way to distinguish a numerically stable computation from an unstable one without external analysis.

This paper describes a proof-of-concept system that eliminates all three problems for a toy transformer language model. Every value in the system is an exact rational number represented as a VDR triple with a fixed denominator of 2^{32} . No floating-point operation occurs anywhere in the pipeline. The computation is deterministic, associative, and carries exact error information in the remainder slot.

The contribution is not a practical replacement for floating-point neural networks. The contribution is a concrete demonstration that exact rational arithmetic can support every operation in an LLM pipeline — and an identification of the specific engineering problems that must be solved to make it work.

2. The VDR Representation

2.1 The Triple

The `vdr-math` library (available on PyPI as `vdr-math`) represents every value as an ordered triple $[V, D, R]$:

- **V** (Value): an arbitrary-precision integer. The settled numerator in the current denominator frame.
- **D** (Denominator): a nonzero integer. The frame in which **V** and **R** are interpreted.
- **R** (Remainder): exact unresolved structure that the denominator frame could not absorb.

A triple $[V, D, 0]$ with $R = 0$ is called *closed* and represents the rational number V/D exactly. A triple with $R \neq 0$ is called *active* and carries additional structure in the remainder slot.

For example, $[3, 7, 0]$ represents $3/7$ exactly. $[1, 3, 1]$ represents $(1 + 1)/3 = 2/3$, where the remainder slot carries the 1 that was not absorbed into V.

The remainder slot is recursive: R can contain an integer base plus a finite list of child VDR triples, each with their own denominator. This forms a finite tree. The tree is always finite — no limits, no approximation, no infinite structures.

2.2 The Fixed-Frame Rule

The central operational rule of VDR arithmetic is: **D never explodes**. When an operation would produce a larger denominator, the system uses `divmod` to keep D fixed and places the exact overflow in R.

Consider multiplying two values on a $D = 2^{32}$ frame:

$A = [p1, 2^{32}, 0]$

$B = [p2, 2^{32}, 0]$

Naive: $[p1 * p2, 2^{64}, 0]$ — D doubled. Never do this.

VDR: $Q, S = \text{divmod}(p1 * p2, 2^{32})$

Result: $[Q, 2^{32}, [S, 2^{32}, 0]]$

D stays at 2^{32} . Q absorbed what fit. S (the remainder) caught what did not. The operation is exact — no information was lost. The `divmod` is a bit shift on powers of two, which is a single machine instruction.

2.3 The Precision Floor

At $D = 2^{32}$, the smallest representable nonzero value is $1/2^{32} \approx 2.33 \times 10^{-10}$. This is the precision floor — the grid spacing of the representation. Every value on the grid is a multiple of this quantity.

When a value is projected onto the grid (for example, $1/3$ projected onto $D = 2^{32}$), the projection rounds to the nearest grid point. The rounding error is at most half the grid spacing: $\approx 1.16 \times 10^{-10}$.

For 2,000 arithmetic operations (approximately what our toy model performs per training epoch), the worst-case accumulated rounding error is $\approx 2,000 \times 2.33 \times 10^{-10} \approx 4.7 \times 10^{-7}$. The observed worst-case error in our experiments was 9.07×10^{-13} , far below the theoretical worst case.

The precision floor is configurable. At $D = 2^{64}$, it drops to $\approx 5.42 \times 10^{-20}$ (19.3 decimal digits). At $D = 2^{128}$, it reaches $\approx 2.94 \times 10^{-39}$ (38.5 decimal digits). The choice of D is a precision-versus-performance knob: smaller D means smaller integers, faster arithmetic, and better hardware alignment; larger D means more precision per grid point.

3. The Denominator Growth Problem

3.1 How D Explodes

Standard rational arithmetic causes denominators to grow. Adding $1/3$ and $1/7$ produces $10/21$ — the denominator went from $\max(3, 7) = 7$ to 21. Multiplying $1/3$ by $1/7$ produces $1/21$. Every cross-denominator operation multiplies the denominators.

In a fixed-frame system where all values start at $D = 2^{32}$, this growth is catastrophic. After 10 multiplications, D reaches 2^{320} — a 97-digit number. The computation has left the fixed frame entirely and is operating in arbitrary-precision territory, which is exactly what the fixed-frame design was meant to avoid.

3.2 Where Growth Occurs in an LLM

We identified six sources of denominator growth in the VDR ML pipeline:

1. **Core arithmetic operators.** The $+$, $-$, $*$, $/$ operators on VDR objects used cross-multiplication ($D1 \times D2$) for operands with different denominators and had no awareness of the basis frame.
2. **Active multiplication.** The active multiplication module (`vdr.active`) patched the core $*$ and $/$ operators with versions that handle active objects ($R \neq 0$), but these patched versions also lacked basis frame awareness.
3. **Mixed-frame operands.** Hyperparameters like learning rate ($VDR(1, 100)$, $D = 100$) and constants like $VDR(2)$ ($D = 1$) mixed with basis-frame values, triggering cross-multiplication.
4. **Softmax Taylor series.** The standard softmax implementation used $\exp(x)$ via Taylor series, which divides by $VDR(k)$ ($D = 1$) at each iteration — 16 mixed-frame divisions per element.
5. **Loss function constants.** Gradient scaling factors like $VDR(2, n)$ had small denominators that mixed with basis-frame gradients.
6. **Autodiff initialization.** Gradient accumulators initialized to $VDR(0)$ ($D = 1$), causing a mixed-frame addition on the first gradient accumulation.

3.3 The Solution: Basis-Aware Operators

We modified the VDR core arithmetic to detect and preserve the basis frame automatically. The solution has three components.

Component 1: Both-in-basis detection. A function `_both_in_basis(a, b)` checks whether both operands share the same D and that D equals the current default basis denominator. When this check passes, the operator uses `divmod` to keep D fixed instead of cross-multiplying.

Component 2: One-in-basis detection. A function `_one_in_basis(a, b)` checks whether exactly one operand is in the basis frame. When this fires, the non-basis operand is projected onto the basis grid before the operation proceeds. This handles all mixed-frame cases — hyperparameters, constants, initial zeros — without requiring the caller to manually project values.

Component 3: Active operator integration. The active multiplication and division module was modified to check for basis-frame operands before falling through to the general active arithmetic path. Active ($R \neq 0$) basis-frame values are flattened to closed form via projection before the basis-frame operation proceeds.

These three checks are inserted at the entry point of every arithmetic operator. They are transparent to the caller — the same code that would have caused D growth now stays in frame automatically. No manual rebase calls are needed anywhere in the model code.

3.4 Verification of Stability

We verified the fix with a test that chains 10 multiplications, 10 divisions, 10 additions, and 10 subtractions at $D = 2^{32}$, checking that D remains exactly 4,294,967,296 after every operation. We also verified that mixing basis-frame values with non-basis values (e.g., $VDR(1, 3)$ with $D = 3$) correctly rebases the non-basis operand. All 54 checks passed.

We then verified stability through the full ML pipeline — forward pass, backward pass, optimizer step, attention, softmax — confirming that all 181 parameters, all intermediate activations, and all logits maintain $D = 2^{32}$ throughout.

4. The Toy Model

4.1 Architecture

The model is a single-block, single-head causal transformer:

```
Token embedding (5  $\times$  4) + Positional embedding (4  $\times$  4)
↓
Self-attention (Wq, Wk, Wv, Wo — each 4  $\times$  4)
↓ + residual connection
Feed-forward network (Linear 4→8, ReLU, Linear 8→4)
↓ + residual connection
Output projection (Linear 4→5)
```

↓

Quadratic softmax surrogate → probability distribution

The vocabulary is 5 tokens: {the, cat, sat, on, mat}. The embedding dimension is 4. The FFN hidden dimension is 8. Context length is 4 tokens. Total trainable parameters: 181.

4.2 Weight Initialization

All weights are initialized as small-integer rationals projected to the $D = 2^{32}$ frame. A deterministic linear congruential generator (LCG) with seed 42 produces integers in $[-4, 4]$, which are stored as $VDR(k, 4)$ and projected via `to_qbasis`. The LCG is:

```
state = (1103515245 * state + 12345) mod 2^31
```

This ensures bit-identical initialization across platforms and runs.

4.3 Quadratic Softmax Surrogate

Standard softmax computes $p_i = \exp(x_i) / \sum \exp(x_j)$. The exponential function is transcendental — it cannot be represented as a finite rational expression. In VDR, it must be approximated via Taylor series, introducing series truncation as a source of controlled error.

We replace softmax entirely with a quadratic surrogate:

$$p_i = (x_i - \text{shift})^2 / \sum_j (x_j - \text{shift})^2$$

where `shift` is the minimum logit. This function involves only subtraction, squaring, summation, and division — all exact rational operations in VDR. No transcendental functions appear anywhere in the forward pass.

The shift parameter serves the same role as max-subtraction in standard softmax: it centers the values to avoid unnecessarily large intermediate results, keeping V slots small in the fixed frame.

To guarantee that probabilities sum to exactly 1, we compute the first $N-1$ probabilities by independent division, then set the last probability to 1 minus their sum. This absorbs all division remainder residuals into the last element. The error in the last element is bounded by $(N-1)/2^{32} \approx 10^{-9}$ for our 5-token vocabulary.

4.4 Training Configuration

- **Loss function:** Mean squared error between softmax output and one-hot target, computed with constants projected to basis frame.
- **Optimizer:** Stochastic gradient descent with learning rate 1/128, projected to basis frame at construction.

- **Corpus:** “the cat sat on the mat” — tokenized to [0, 1, 2, 3, 0, 4], producing 2 training windows of length 4.
- **Epochs:** 20.
- **Backward pass:** Manual backpropagation through output projection, FFN (with ReLU derivative), residual connections, attention block (with surrogate softmax backward), and projection matrices. All gradients exact.

4.5 Surrogate Softmax Backward

The backward pass through the quadratic surrogate is simpler than through exponential softmax. For the forward function $p_i = s_i^2 / \sum s_j^2$ where $s_i = x_i - \text{shift}$:

$$\partial L / \partial s_i = (2 * s_i / \sum s_j^2) * (\partial L / \partial p_i - \sum_j \partial L / \partial p_j * p_j)$$

This involves only the cached shifted values, the cached probabilities, and the incoming gradient — all exact rational operations. No derivative of exp is needed.

4.6 Text Generation

Four decoding strategies are implemented:

- **Greedy:** Select the token with highest probability (exact argmax comparison).
- **Categorical sampling:** Build exact rational CDF, compare against a deterministic rational random value in [0, 1).
- **Top-k:** Keep k highest probabilities, renormalize exactly, then categorical sample.
- **Nucleus (top-p):** Keep smallest set exceeding threshold, renormalize exactly, then categorical sample.

All sampling decisions are exact rational comparisons. There are no float threshold ambiguities — the CDF has no gaps or overlaps because the probabilities sum to exactly 1.

5. Results

5.1 Training

The model trains successfully with monotonically decreasing loss:

Epoch	Loss	Softmax sum = 1
1	0.267299	yes
5	0.255813	yes
10	0.244555	yes
15	0.234526	yes

Epoch	Loss	Softmax sum = 1
20	0.225423	yes

The softmax sum check passes at every epoch within a tolerance of 10^{-9} . The worst observed residual across all epochs was 9.07×10^{-13} at epoch 12 — a single instance where division remainder accumulation in the surrogate produced a sum that differed from 1 by less than one part in 10^{12} .

5.2 Attention Pattern

The learned attention pattern for the context “the cat sat on”:

	the	cat	sat	on
the	1.000	0.000	0.000	0.000
cat	0.500	0.500	0.000	0.000
sat	0.333	0.335	0.332	0.000
on	0.283	0.527	0.000	0.190

Causal masking is exact: future positions receive exactly zero attention weight, not an approximation of zero. Each row sums to exactly 1.

5.3 Generation

After training, greedy decoding from the prompt “the cat”:

the cat sat sat sat sat

The model has learned that “sat” is the most probable next token in most contexts, which is consistent with the training data (“the cat sat on the mat” provides “sat” as the target for the “the cat” context).

5.4 Verification

Nine tests, all passing:

Test	Description	Result
softmax sum	Every probability vector sums to 1 (within 10^{-9})	PASS
attention weights	Every attention weight row sums to 1	PASS

Test	Description	Result
d stability	All 181 parameters and all logits have $D = 2^{32}$	PASS
deterministic	Two runs from seed 42 produce bit-identical losses and weights	PASS
forward backward roundtrip	Same init, one training step, bit-identical results	PASS
checkpoint roundtrip	Save $\rightarrow$ perturb $\rightarrow$ restore $\rightarrow$ bit-identical	PASS
weight update	$(w_{\text{old}} - w_{\text{new}}) / \text{lr} == \text{grad}$, exact equality	PASS
loss monotonicity	Loss at epoch 20 $<$ loss at epoch 1	PASS
gradient correctness	Analytical gradient matches finite difference ($\text{diff} < 0.1$)	PASS

The deterministic and roundtrip tests are the strongest results. They demonstrate that the computation produces identical bit patterns across runs — a property that is structurally impossible with floating-point arithmetic due to non-associative addition.

5.5 Denominator Report

all 181 parameters have $D = 4294967296$ [OK]

No denominator growth was observed at any point during training, generation, or verification.

6. Discussion

6.1 What This Demonstrates

This toy model demonstrates five things:

1. **Every operation in an LLM pipeline can be expressed in fixed-denominator rational arithmetic.** Forward pass, backpropagation, attention, softmax, sampling, weight updates — all work without floating-point.
2. **The denominator growth problem is solvable at the operator level.** By making the core arithmetic operators basis-aware, frame stability becomes automatic. Model code does not need to manage frames explicitly.

3. **The quadratic softmax surrogate eliminates transcendentals from the forward pass.** This removes the most expensive non-polynomial operation from the pipeline and simplifies the backward pass.
4. **Exact determinism is achievable.** Same seed produces bit-identical results across runs. This is a structural property of the arithmetic, not a coincidence of implementation.
5. **$D = 2^{32}$ provides sufficient precision for a toy model.** The precision floor of $\sim 10^{-10}$ per operation supports 20 epochs of training with worst-case accumulated error well below 10^{-6} .

6.2 What This Does Not Demonstrate

This model is a proof of concept with deliberate limitations:

- **Scale.** 181 parameters and 5 tokens. Production LLMs have billions of parameters and vocabularies of 50,000+. The computational cost of VDR arithmetic is $\sim 50\text{--}200 \times$ float per operation in Python, making direct scaling impractical without compiled implementations.
- **Convergence quality.** The quadratic surrogate produces different optimization landscapes than exponential softmax. The model converges, but we have not compared convergence rates or final loss quality against a float baseline at the same scale.
- **Layer normalization.** The model omits layer norm, which requires reciprocal square root — an irrational operation that must be approximated via Newton iteration in VDR. Adding it is engineering work, not a mathematical obstacle.
- **Multi-head attention.** The model uses a single head. Multi-head attention involves splitting and concatenating vectors, which are structurally trivial in VDR, but the increased number of operations would test D stability more thoroughly.
- **Hardware acceleration.** All arithmetic runs in Python on arbitrary-precision integers. A production implementation would require compiled kernels operating on fixed-width integers matching the D frame (32-bit or 64-bit values for $D = 2^{32}$ or 2^{64}).

6.3 The Precision-Performance Tradeoff

The choice of D directly controls both precision and performance:

D	Decimal digits	$V \times V$ product width	Hardware fit
2^{16}	~ 4.8	32-bit	16-bit DSP, embedded
2^{32}	~ 9.6	64-bit	Standard 32-bit ALU
2^{64}	~ 19.3	128-bit	SIMD, GPU registers
2^{128}	~ 38.5	256-bit	SIMD with widening

At $D = 2^{32}$, the product of two V slots is a 64-bit integer. The `divmod` by 2^{32} is a 32-bit right shift — one instruction. This makes $D = 2^{32}$ naturally aligned with 32-bit and 64-bit hardware.

At $D = 2^{64}$, the product is a 128-bit integer and the `divmod` is a 64-bit shift. Modern SIMD instruction sets (AVX-512, NEON) support 128-bit integer operations, making this feasible in compiled code.

6.4 Comparison to Quantized Inference

Current quantized inference methods (INT8, INT4, GPTQ, AWQ) share a structural similarity with VDR: they represent values as integers with a scaling factor. The key difference is error tracking.

Quantized inference discards the quantization error. When a float32 weight is quantized to INT8, the rounding error is lost — there is no record of what was discarded, and no mechanism to recover it.

VDR's remainder slot catches the exact overflow from every operation. The remainder could, in principle, be monitored (to detect precision degradation), accumulated (to periodically correct the main value), or carried through the computation (at the cost of tree depth growth). This toy model flattens remainders at every operation (which is what `_basis_mul` does via `to_qbasis`), discarding them like quantization does. But the infrastructure for exact carry exists in the library and could be used selectively in precision-critical paths.

7. Next Steps

7.1 Compiled Arithmetic Kernels

The immediate bottleneck is Python's overhead on integer arithmetic. A C or Rust implementation of basis-frame multiply and divide — operating on fixed-width 32-bit or 64-bit V slots with bit-shift `divmod` — would bring per-operation cost close to integer quantized inference. The VDR library's basis-aware logic is simple enough to compile: check if D matches, multiply V slots, shift right, store quotient and remainder.

7.2 GPU Implementation

At $D = 2^{32}$, the basis-frame multiply is: multiply two 32-bit integers to get a 64-bit product, split into upper 32 bits (quotient) and lower 32 bits (remainder). This maps directly to GPU integer multiply-and-split instructions. A CUDA kernel performing this operation across tensor elements would enable batch inference at near-INT32 throughput.

7.3 Larger Models

Scaling to larger models requires verifying D stability across deeper operation chains (96+ layers), wider hidden dimensions (4096+), and longer sequences (4096+ tokens). The

operator-level basis-awareness demonstrated here should generalize, but accumulation of precision-floor errors over longer chains needs empirical measurement at each frame size.

7.4 Selective Remainder Tracking

The current implementation flattens all remainders at every operation. An alternative is to carry remainders through precision-sensitive paths (e.g., residual stream accumulation across layers) and flatten only at precision-insensitive boundaries (e.g., after softmax normalization). This would provide exact-carry benefits where they matter most without the cost of full tree-depth growth.

7.5 Mixed-Precision Frames

Different parts of the model could use different D values — $D = 2^{16}$ for weight storage (matching INT16 quantization), $D = 2^{32}$ for accumulation, $D = 2^{64}$ for gradient statistics. The basis-aware operators already handle rebasing between frames. The engineering task is determining the optimal frame assignment per layer boundary.

8. Conclusion

We built a toy transformer language model that trains and generates text using exact rational arithmetic with a fixed denominator of 2^{32} . The model demonstrates that every operation in an LLM pipeline — embedding lookup, matrix multiplication, attention score computation, softmax probability normalization, backpropagation, and stochastic gradient descent — can be performed without floating-point arithmetic while maintaining denominator stability.

The key engineering contributions are basis-aware arithmetic operators that automatically prevent denominator growth, and a quadratic softmax surrogate that eliminates transcendental functions from the pipeline. The key empirical result is that 181 parameters trained for 20 epochs maintain $D = 2^{32}$ throughout, with bit-identical determinism across runs and softmax outputs summing to 1 within 10^{-12} .

The system is a proof of concept, not a production tool. But it establishes that fixed-denominator rational arithmetic is a viable substrate for neural network computation, and identifies a concrete path from toy demonstration to compiled, hardware-accelerated implementation.

Appendix A: Software

The `vdr-math` library is available on PyPI:

```
pip install vdr-math
```

The toy LLM source code is included in the library repository at `examples/toy_llm/`. To run:

```
cd examples/toy_llm  
  
python run.py all
```

All code is Python 3.8+ with no external dependencies. The library is MIT licensed.

Appendix B: Verification Test Details

softmax_sum. For each training window, computes the forward pass, applies the quadratic surrogate, and sums the resulting probability vector. Checks that the sum equals 1 within 10^{-9} . The tolerance accounts for division remainder residuals at $D = 2^{32}$: with $N = 5$ probabilities, the maximum residual from divmod truncation is $(N-1)/2^{32} \approx 9.3 \times 10^{-10}$.

attention_weights. For each training window, runs the attention block and checks that every weight row sums to exactly 1. Uses the same sum-to-one construction as the output softmax.

d_stability. Iterates over all 181 parameters and all logit vectors after a forward pass, checking that every VDR element has $D = 4,294,967,296$.

deterministic. Instantiates two models from seed 42, trains each for 3 epochs, and compares loss values and parameter values. Uses VDR value equality (exact rational comparison), not float comparison.

forward_backward_roundtrip. Instantiates two models from seed 42, performs one training step on each with the same data, and compares all parameter values for exact equality.

checkpoint_roundtrip. Saves all parameter values, perturbs every parameter by adding 1 (in basis frame), restores from saved values, and verifies exact match.

weight_update. Saves a weight value before one SGD step, computes the step, and verifies that $(w_{\text{old}} - w_{\text{new}})$ equals $\text{lr} \times \text{grad}$ exactly.

loss_monotonicity. Trains for 10 epochs and checks that the final epoch loss is strictly less than the first epoch loss.

gradient_correctness. Computes the analytical gradient of the loss with respect to one weight element, then computes the numerical gradient via finite difference with step size $1/10000$ in basis frame. Checks that the difference is less than 0.1. The finite-difference gradient is a discrete derivative, not a limit, so exact agreement is not expected — the test verifies directional consistency.

Appendix C: Operator Basis-Awareness Coverage

The following table documents every arithmetic path in the VDR core and active modules, whether it checks the basis frame, and what action it takes when the check fires.

Operator	Module	Both-in-basis action	One-in-basis action	Neither action
add	core.py	Integer add, same D	Rebase non-basis, integer add	Cross-multiply $D1 \times D2$
sub	core.py	Integer subtract, same D	Rebase non-basis, integer subtract	Cross-multiply $D1 \times D2$
mul	core.py	basis mul (divmod)	—	Cross-multiply $D1 \times D2$
truediv	core.py	basis div (divmod)	—	Reciprocal multiply
active mul	active.py	Flatten active, basis mul	Rebase non-basis, basis mul	Cross-multiply with remainder tree
active div	active.py	Flatten active, basis div	Rebase non-basis, basis div	Project divisor to Fraction, reciprocal multiply
active add	core.py	Integer add (same-D path)	Rebase non-basis, integer add	Cross-multiply $D1 \times D2$, lift remainders

The core `__mul__` and `__truediv__` only check `_both_in_basis`, not `_one_in_basis`. This is because `active.install()` patches these operators with `active_mul` and `active_div`, which perform both checks. The core versions are fallbacks that only execute if `active.install()` has not been called.

Appendix D: D Growth Under Original Operators

Before the basis-aware fix, chaining 10 multiplications at $D = 2^{32}$ produced the following denominator growth:

Step	D (bit length)	D (approximate)
0	33	4.3×10^9
1	65	1.8×10^{19}
2	97	7.9×10^{28}

Step	D (bit length)	D (approximate)
3	129	3.4×10^{38}
4	161	1.5×10^{48}
5	193	6.3×10^{57}
6	225	2.7×10^{67}
7	257	1.2×10^{77}
8	289	5.0×10^{86}
9	321	2.1×10^{96}
10	353	9.2×10^{105}

Each multiplication doubled the bit length of D. After 10 steps, D exceeded 10^{105} — a number with more digits than the number of atoms in the observable universe. At this scale, every arithmetic operation requires arbitrary-precision integer multiplication on 300+ bit numbers, eliminating any performance advantage of the fixed-frame design.

Division produced a different growth pattern. Because VDR division inverts the divisor (swapping V and D), the resulting denominator depends on the V slot of the divisor, which changes unpredictably:

Step	D
0	4,294,967,296
1	613,566,757
2	376,464,165,295,497,049
3	230,985,897,027,070,071,058,000,093
4	1.4×10^{35}
5	8.7×10^{43}

The growth is slower than multiplication (roughly 8-9 digits per step instead of 10) but still exponential.

Addition between same-D operands never caused growth — the same-D fast path was always present. Addition between different-D operands (e.g., basis value + VDR(1, 3)) produced $D = D1 \times D2$, which is a one-time $3 \times$ growth, not exponential — but it moved the result off the power-of-two grid permanently.

After the basis-aware fix, all operations maintain $D = 4,294,967,296$ exactly. The growth table for 10 chained multiplications under the fix:

Step	D
1–10	4,294,967,296

Appendix E: Mixed-Frame Sources in the ML Pipeline

The following table catalogs every location in the ML modules where a non-basis VDR constant would have mixed with basis-frame values prior to the rewrite. Each entry shows the original constant, its D value, where it appeared, and the fix applied.

Module	Function	Original constant	Original D	Context	Fix
softmax.py	exp	VDR (k) for k=1..16	1	Taylor term division	Precomputed 1/k table in basis via to qbasis
softmax.py	softmax surrogate square	(no shift parameter)	—	Raw logit squaring	Added shift parameter, default to min logit
attention.py	apply boolean mask	VDR (-1000)	1	Mask fill value	Project fill to basis once via basis fill value
losses.py	mse	VDR (n) divisor	n	Loss normalization	Precomputed VDR (1, n) in basis via basis const
losses.py	mse grad	VDR (2, n)	n	Gradient scaling	Precomputed in basis via basis const
losses.py	l1 grad	VDR (0)	1	Zero gradient branch	Replaced with basis const (0)
losses.py	hinge binary	VDR (1), VDR (label)	1	Margin computation	Projected via basis const
losses.py	cross entropy binary	VDR (1)	1	Complement probability	Projected via basis const
optim.py	SGD .init	lr (arbitrary D)	varies	Learning rate	Projected to basis at construction

Module	Function	Original constant	Original D	Context	Fix
optim.py	Momentum.init	lr, beta	varies	Hyperparameters	Projected to basis at construction
optim.py	Momentum.VDR	1	1	Gradient coefficient	Cached as self.one in basis
autodiff.py	Node.init	VDR(0) grad	1	Initial gradient	basis zero()
autodiff.py	Node.backward	VDR(1) seed	1	Backward seed	basis vdr(VDR(1))
autodiff.py	ensure node	VDR(x) from int	1	Constant wrapping	basis vdr(VDR(x))
autodiff.py	pow	VDR(exp)	1	Exponent constant	basis vdr(VDR(exp))
autodiff.py	relu	VDR(0)	1	Zero output	basis zero()
autodiff.py	mse loss	VDR(1, n)	n	Loss scaling	basis vdr(VDR(1, n))
tensor.py	Tensor3D.VDR	0	1	Zero fill	basis zero vec(d)
tensor.py	masked fill rows	VDR(0) fill	1	Mask fill	basis zero() or projected fill
transformer.py	TransformerBlock	Wq, Wk, Wv, Wo	varies	Weight matrices	to qbasis via TransformerBlock.to_qbasis()
transformer.py	TransformerBlock	output proj weights	varies	Output layer	self.output proj.to_qbasis(bits)
nn.py	ReLU.forward	VDR(0) comparison	1	Threshold	Comparison only, no arithmetic mixing

Total: 23 mixed-frame sources identified and fixed across 8 modules.

Appendix F: Precision Floor by Frame Size

Bits	D	Decimal digits	Floor (per op)	After 10 ³ ops	After 10 ⁶ ops	After 10 ⁹ ops	Hardware alignment
8	256	2.4	3.9×10^{-3}	3.9	—	—	Byte
16	65,536	4.8	1.5×10^{-5}	1.5×10^{-2}	15.3	—	INT16
32	4.3×10^9	9.6	2.3×10^{-10}	2.3×10^{-7}	2.3×10^{-4}	0.23	INT32
64	1.8×10^{19}	19.3	5.4×10^{-20}	5.4×10^{-17}	5.4×10^{-14}	5.4×10^{-11}	INT64, SIMD
128	3.4×10^{38}	38.5	2.9×10^{-39}	2.9×10^{-36}	2.9×10^{-33}	2.9×10^{-30}	AVX-512
335	8.7×10^{100}	100.9	1.1×10^{-101}	1.1×10^{-98}	1.1×10^{-95}	1.1×10^{-92}	Arbitrary precision

The “After N ops” columns show worst-case accumulated error assuming every operation contributes maximum rounding error in the same direction. In practice, errors partially cancel, so observed accumulation is significantly less. Our toy model at $D = 2^{32}$ performed ~2,000 operations per epoch and observed worst-case accumulated error of 9.07×10^{-13} , compared to the theoretical worst case of $\sim 4.7 \times 10^{-7}$.

For reference, float32 has ~7.2 decimal digits of precision and float64 has ~15.9. The $D = 2^{32}$ frame (9.6 digits) sits between them. The $D = 2^{64}$ frame (19.3 digits) exceeds float64 precision.

Appendix G: Softmax Surrogate Sum Residuals

The following table shows the worst softmax sum residual observed at each epoch during the 20-epoch training run. The residual is $|\sum p_i - 1|$ measured over all training windows in that epoch.

Epoch	Worst residual	Passes 10 ⁻⁹ threshold
1	0	yes
2	0	yes
3	0	yes

Epoch	Worst residual	Passes 10^{-9} threshold
4	0	yes
5	0	yes
6	0	yes
7	0	yes
8	0	yes
9	0	yes
10	0	yes
11	0	yes
12	9.07×10^{-13}	yes
13	0	yes
14	0	yes
15	0	yes
16	0	yes
17	0	yes
18	0	yes
19	0	yes
20	0	yes

The epoch 12 residual is the only nonzero entry. It occurs because the one - running correction in `softmax_surrogate_square` guarantees the sum structurally, but the probabilities then pass through subsequent arithmetic (MSE computation, gradient scaling) before the training loop’s sum check re-sums them from the returned values. The re-summation accumulates basis-frame division residuals that were not present in the original structurally-guaranteed sum.

This is a measurement artifact, not a computation error: the softmax output itself sums to exactly 1, but summing the same values again through a different addition order can produce a different structural representation of the same rational value. The Fraction-based value comparison confirms the sum is exactly 1; the float conversion introduces the apparent 10^{-13} discrepancy.

Appendix H: Toy Model Parameter Census

Component	Parameter	Shape	Elements	Frame
Token embedding	token emb	5×4	20	$D = 2^{32}$
Positional embedding	pos emb	4×4	16	$D = 2^{32}$
Query projection	Wq.weight	4×4	16	$D = 2^{32}$
Query projection	Wq.bias	4	4	$D = 2^{32}$
Key projection	Wk.weight	4×4	16	$D = 2^{32}$
Key projection	Wk.bias	4	4	$D = 2^{32}$
Value projection	Wv.weight	4×4	16	$D = 2^{32}$

Component	Parameter	Shape	Elements	Frame
Value projection	Wv.bias	4	4	$D = 2^{32}$
Output projection	Wo.weight	4×4	16	$D = 2^{32}$
Output projection	Wo.bias	4	4	$D = 2^{32}$
FFN layer 1	ffn l1.weight	8×4	32	$D = 2^{32}$
FFN layer 1	ffn l1.bias	8	8	$D = 2^{32}$
FFN layer 2	ffn l2.weight	4×8	32	$D = 2^{32}$
FFN layer 2	ffn l2.bias	4	4	$D = 2^{32}$
Output head	output.weight	5×4	20	$D = 2^{32}$
Output head	output.bias	5	5	$D = 2^{32}$
Total			217	

Note: the denominator report in the verification suite reports 181 parameters because it counts only the `parameters()` list returned by the model, which excludes the non-trainable embedding tables (36 elements). The full model contains 217 VDR values, all at $D = 2^{32}$.

Appendix I: Operation Count Per Forward Pass

The following table estimates the number of VDR arithmetic operations for a single forward pass with sequence length 4 and the toy model dimensions.

Stage	Operation	Count per call	Calls	Total ops
Embedding	Vec add (token + pos)	4 adds	4 positions	16
Q projection	Mat-vec ($4 \times 4 \cdot 4$)	16 mul + 12 add	4 positions	112
K projection	Mat-vec ($4 \times 4 \cdot 4$)	16 mul + 12 add	4 positions	112
V projection	Mat-vec ($4 \times 4 \cdot 4$)	16 mul + 12 add	4 positions	112
Attention scores	Vec dot (4-dim)	4 mul + 3 add	10 (causal)	70
Surrogate softmax	Square + sum + div	~3n per row	4 rows	~60
Value mix	Weighted sum	~8 per output elem	4×4	~128
Wo projection	Mat-vec ($4 \times 4 \cdot 4$)	16 mul + 12 add	4 positions	112
Residual add	Vec add	4 adds	4 positions	16
FFN layer 1	Mat-vec ($8 \times 4 \cdot 4$)	32 mul + 24 add	4 positions	224

Stage	Operation	Count per call	Calls	Total ops
ReLU	Comparison	8 comparisons	4 positions	32
FFN layer 2	Mat-vec ($4 \times 8 \cdot 8$)	32 mul + 28 add	4 positions	240
Residual add	Vec add	4 adds	4 positions	16
Output projection	Mat-vec ($5 \times 4 \cdot 4$)	20 mul + 15 add	4 positions	140
Total				~1,390

The backward pass performs approximately the same number of operations (gradient computation mirrors forward computation plus outer products for weight gradients), bringing the total per training step to approximately 2,800 operations.

Over 20 epochs with 2 training windows each: $20 \times 2 \times 2,800 \approx 112,000$ total VDR operations during training.

Appendix J: Comparison of Softmax Approaches in VDR

Property	Taylor exp softmax	Quadratic surrogate
Forward ops per element	~32 (16 mul + 16 div for depth 16)	3 (subtract, square, divide)
Backward complexity	Chain rule through 16 Taylor terms	Single closed-form expression
Transcendental functions	Yes (exp approximated)	None
Series truncation error	Yes (controlled by depth)	None
Distribution shape	Sharply peaked (exponential)	Broadly spread (quadratic)
Sum-to-one guarantee	By construction after correction	By construction after correction
Gradient saturation	Yes (extreme logits $\rightarrow$ near-zero grad)	No (gradient proportional to input)
Mixed-frame constants	16 VDR (k) divisions per element	1 shift subtraction
Suitability for VDR	Functional but expensive	Natural fit

The quadratic surrogate is not an approximation of exponential softmax. It is a different normalization function that happens to satisfy the same structural requirements: non-negative outputs, sum to 1, differentiable. The optimization landscape is different — the quadratic surrogate produces broader probability distributions and does not suffer from gradient saturation at extreme logit values. Whether this is advantageous depends on the task and scale.

Appendix K: Verification Test Execution Order and Dependencies

Test	Requires trained model	Creates own model	Modifies model state	Runtime (approx)
softmax sum	No (uses provided)	If none provided	No (forward only)	< 0.1s
attention weights	No (uses provided)	If none provided	No (forward only)	< 0.1s
d stability	No (uses provided)	If none provided	No (forward only)	< 0.1s
deterministic	No	Yes (2 models)	Yes (trains both)	~2s
forward backward roundtrip	No	Yes (2 models)	Yes (1 step each)	~0.5s
checkpoint roundtrip	No (uses provided)	If none provided	Yes (perturb + restore)	< 0.1s
weight update	No	Yes	Yes (1 step)	~0.5s
loss	No	Yes	Yes (10 epochs)	~3s
monotonicity gradient correctness	No	Yes	Yes (forward + perturb)	~0.5s

Tests are ordered to run cheap structural checks first (softmax_sum, attention_weights, d_stability), then determinism checks (which require full training runs), then single-step correctness checks. The total verification suite runs in approximately 7 seconds on a 2019 laptop.

The `checkpoint_roundtrip` test is the only one that modifies the provided model (perturbs then restores). All other tests either use the model read-only or create their own instances. The restore step is verified to be bit-identical to the original state, so the provided model is not permanently altered.

[[HOWL-VDR-31-2026](#)] VDR Toy LLM. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-31-2026>

[[HOWL-VDR-1-2026](#)] VDR Arithmetic: Value, Decimal, Remainder. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-1-2026>

[[HOWL-VDR-2-2026](#)] VDR Gym: Exact Arithmetic Across Fifteen Domains. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-2-2026>

[[HOWL-MATH-3-2026](#)] The Transcendental Hierarchy. Github: <https://github.com/ghowland/papers/tree/main/papers/M>

MATH-3-2026

[[HOWL-MATH-4-2026](#)] The Universal Power-of-Two Basis. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-4-2026>

[[HOWL-VDR-14-2026](#)] VDR-LLM-Prolog. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-14-2026>

[[HOWL-VDR-21-2026](#)] VDR-LLM-Prolog on FPGA. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-21-2026>

[[HOWL-VDR-22-2026](#)] VDR-LLM-Prolog on Dedicated Silicon. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-22-2026>

[[HOWL-VDR-23-2026](#)] VDR-LLM-Prolog: Functional Remainder Hardware. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-23-2026>

[[HOWL-VDR-24-2026](#)] LLM Software. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-24-2026>

[[HOWL-VDR-25-2026](#)] LLM Server Software. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-25-2026>

[[HOWL-VDR-26-2026](#)] VDR and Diffusion. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-26-2026>

[[HOWL-VDR-27-2026](#)] VDR Beyond Language Models. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-27-2026>

[[HOWL-VDR-28-2026](#)] Exact Rational Arithmetic for Sequential Computation. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-28-2026>

[[HOWL-VDR-29-2026](#)] VDR in Zig SIMD and GPU Performance versus Floating Point. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-29-2026>

[[HOWL-VDR-30-2026](#)] Economics of Scale: Floating Point vs Exact Integer ML Models. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-30-2026>